

# 1. Binary Tree Height Calculation Challenges

- Write a C program to calculate the height of a binary tree. Ensure the program handles empty trees gracefully.

```
// Structure for a binary tree node
struct TreeNode {
    int data;
    struct TreeNode* left;
    struct TreeNode* right;
};
```

```
1 // Including necessary header files
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 // Structure for a binary tree node
6 struct TreeNode {
7     int data;
8     struct TreeNode* left;
9     struct TreeNode* right;
10 };
11
12 // Function to create a new node
13 struct TreeNode* createNode(int value) {
14     struct TreeNode* newNode = (struct TreeNode*)malloc(sizeof(struct TreeNode));
15     if (newNode != NULL) {
16         newNode->data = value;
17         newNode->left = NULL;
18         newNode->right = NULL;
19     }
20     return newNode;
21 }
22
```

```
23 // Function to insert a node into the binary tree
24 struct TreeNode* insertNode(struct TreeNode* root, int value) {
25     if (root == NULL) {
26         return createNode(value);
27     }
28
29     if (value < root->data) {
30         root->left = insertNode(root->left, value);
31     } else if (value > root->data) {
32         root->right = insertNode(root->right, value);
33     }
34
35     return root;
36 }
37
38 // Function to calculate the height of a binary tree
39 int treeHeight(struct TreeNode* root) {
40     if (root == NULL) {
41         return 0; // Height of an empty tree is 0
42     }
43
44     // Recursively calculate the height of the left and right subtrees
45     int leftHeight = treeHeight(root->left);
46     int rightHeight = treeHeight(root->right);
47
48     // Return the maximum height of the left and right subtrees, plus 1 for the current node
49     return (leftHeight > rightHeight) ? (leftHeight + 1) : (rightHeight + 1);
50 }
51
```

```

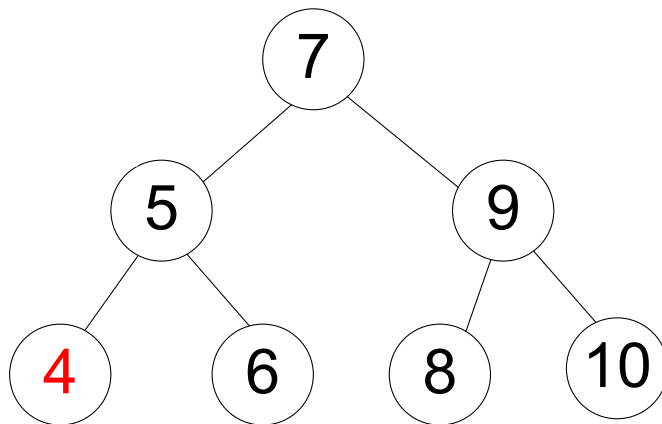
52 // Function to free the memory allocated for the binary tree
53 void freeTree(struct TreeNode* root) {
54     if (root != NULL) {
55         freeTree(root->left);
56         freeTree(root->right);
57         free(root);
58     }
59 }
60
61 int main() {
62     struct TreeNode* root = NULL;
63     int nodeValue;
64     char choice;
65
66     // Insert nodes into the binary tree
67     do {
68         printf("Input a value to insert into the binary tree (enter 0 to stop): ");
69         scanf("%d", &nodeValue);
70
71         if (nodeValue != 0) {
72             root = insertNode(root, nodeValue);
73         }
74
75     } while (nodeValue != 0);
76
77     // Calculate and display the height of the binary tree
78     int height = treeHeight(root);
79     printf("\nHeight of the Binary Tree: %d\n", height);
80
81     // Free allocated memory
82     freeTree(root);
83
84     return 0;
85 }

```

## 2. Root-to-Leaf Path Sum Challenges

- Write a C program to determine if a binary tree has a root-to-leaf path whose sum equals a given target sum.

```
// Structure for a binary tree node
struct TreeNode {
    int data;
    struct TreeNode* left;
    struct TreeNode* right;
};
```



Case 1: Target Sum = 16

There exists a root-to-leaf path (7->5->4) whose sum equals 16

Case 2: Target Sum = 80

There does not exist a root-to-leaf path whose sum equals 80

```
1 // Including necessary header files
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <stdbool.h>
5
6 // Structure for a binary tree node
7 struct TreeNode {
8     int data;
9     struct TreeNode* left;
10    struct TreeNode* right;
11 };
12
13 // Function to create a new node
14 struct TreeNode* createNode(int value) {
15     struct TreeNode* newNode = (struct TreeNode*)malloc(sizeof(struct TreeNode));
16     if (newNode != NULL) {
17         newNode->data = value;
18         newNode->left = NULL;
19         newNode->right = NULL;
20     }
21     return newNode;
22 }
23
```

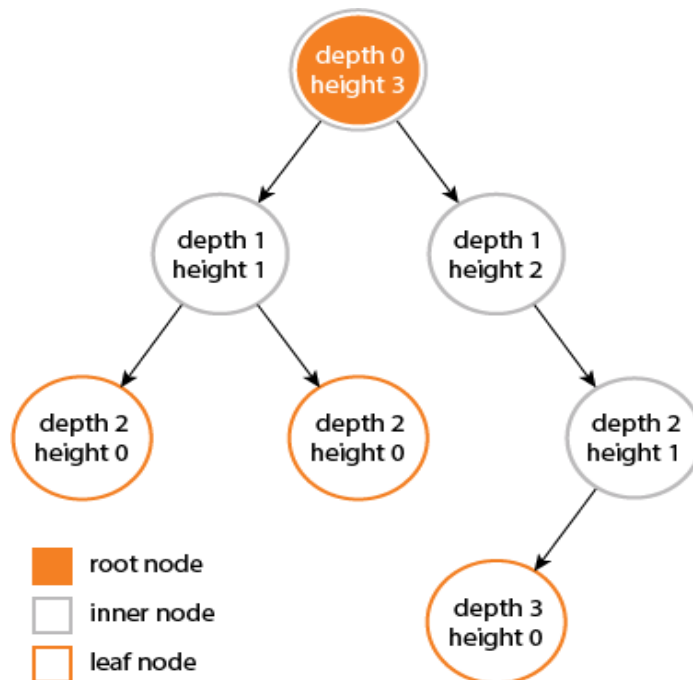
```
24 // Function to check if a path with the given sum exists in the binary tree
25 bool hasPathSum(struct TreeNode* root, int targetSum) {
26     if (root == NULL) {
27         return false;
28     }
29
30     // Check if the current node is a leaf
31     if (root->left == NULL && root->right == NULL) {
32         return (targetSum - root->data) == 0;
33     }
34
35     // Recursively check the left and right subtrees
36     bool leftPath = hasPathSum(root->left, targetSum - root->data);
37     bool rightPath = hasPathSum(root->right, targetSum - root->data);
38
39     return leftPath || rightPath;
40 }
41
42 // Function to free the memory allocated for the binary tree
43 void freeTree(struct TreeNode* root) {
44     if (root != NULL) {
45         freeTree(root->left);
46         freeTree(root->right);
47         free(root);
48     }
49 }
50
```

```
51 int main() {
52     // Build a sample binary tree
53     struct TreeNode* root = createNode(5);
54     root->left = createNode(4);
55     root->right = createNode(8);
56     root->left->left = createNode(11);
57     root->left->left->left = createNode(7);
58     root->left->left->right = createNode(2);
59     root->right->left = createNode(13);
60     root->right->right = createNode(4);
61     root->right->right->right = createNode(1);
62
63     // Set the target sum
64     int targetSum = 18;
65     // Check if a path with the given sum exists
66     if (hasPathSum(root, targetSum)) {
67         printf("Path with sum %d exists in the binary tree.\n", targetSum);
68     } else {
69         printf("No path with sum %d exists in the binary tree.\n", targetSum);
70     }
71
72     // Free allocated memory
73     freeTree(root);
74
75     return 0;
76 }
```



# 3. Depth-Order Traversal Variants

- Write a C program that extends the binary tree program to perform a depth-order traversal (also called level-order traversal). Print the nodes in the ascending order of the depth (e.g. from depth 0 to depth 3 in the tree below).



```
// Structure for a binary tree node
struct TreeNode {
    int data;
    struct TreeNode* left;
    struct TreeNode* right;
};
```

```
1 // Including necessary header files
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <stdbool.h>
5
6 // Structure for a binary tree node
7 struct TreeNode {
8     int data;
9     struct TreeNode* left;
10    struct TreeNode* right;
11 };
12
13 // Function to create a new node
14 struct TreeNode* createNode(int value) {
15     struct TreeNode* newNode = (struct TreeNode*)malloc(sizeof(struct TreeNode));
16     if (newNode != NULL) {
17         newNode->data = value;
18         newNode->left = NULL;
19         newNode->right = NULL;
20     }
21     return newNode;
22 }
23
```

```
24 // Function to insert a node into the binary tree
25 struct TreeNode* insertNode(struct TreeNode* root, int value) {
26     if (root == NULL) {
27         return createNode(value);
28     }
29
30     if (value < root->data) {
31         root->left = insertNode(root->left, value);
32     } else if (value > root->data) {
33         root->right = insertNode(root->right, value);
34     }
35
36     return root;
37 }
38
39 // Function to perform in-order traversal and print elements in sorted order
40 void inOrderTraversal(struct TreeNode* root) {
41     if (root != NULL) {
42         inOrderTraversal(root->left);
43         printf("%d ", root->data);
44         inOrderTraversal(root->right);
45     }
46 }
47
```

```
48 // Function to perform level-order traversal and print nodes at each level
49 void levelOrderTraversal(struct TreeNode* root) {
50     if (root == NULL) {
51         printf("The tree is empty.\n");
52         return;
53     }
54
55     // Create a queue for level-order traversal
56     struct TreeNode* queue[1000];
57     int front = -1, rear = -1;
58
59     // Enqueue the root node
60     queue[++rear] = root;
61
```

```
62     while (front != rear) {
63         // Dequeue a node and print its value
64         struct TreeNode* current = queue[++front];
65         printf("%d ", current->data);
66
67         // Enqueue the left child if exists
68         if (current->left != NULL) {
69             queue[++rear] = current->left;
70         }
71
72         // Enqueue the right child if exists
73         if (current->right != NULL) {
74             queue[++rear] = current->right;
75         }
76     }
77
78     printf("\n");
79 }
80
```

```
81 // Function to free the memory allocated for the binary tree
82 void freeTree(struct TreeNode* root) {
83     if (root != NULL) {
84         freeTree(root->left);
85         freeTree(root->right);
86         free(root);
87     }
88 }
89
```

```
90 int main() {
91     struct TreeNode* root = NULL;
92     int nodeValue;
93     char choice;
94
95     // Insert nodes into the binary tree
96     do {
97         printf("Input a value to insert into the binary tree (enter 0 to stop): ");
98         scanf("%d", &nodeValue);
99
100        if (nodeValue != 0) {
101            root = insertNode(root, nodeValue);
102        }
103
104    } while (nodeValue != 0);
105
106    // Perform in-order traversal and print elements in sorted order
107    printf("\nIn-order Traversal (Sorted Order) of the Binary Tree: ");
108    inOrderTraversal(root);
109    printf("\n");
110
111    // Perform level-order traversal and print nodes at each level
112    printf("\nLevel-order Traversal of the Binary Tree:\n");
113    levelOrderTraversal(root);
114
115    // Free allocated memory
116    freeTree(root);
117
118    return 0;
119 }
```